



CC65 Programación concurrente y distribuida

Enunciado del Trabajo Final del Curso

Profesores:	Carlos Alberto Jara García / Javier Antonio Prudencio Vidal
Sección:	Todas
Fecha de evaluación:	Semana 15
Ciclo académico:	2026-10

Introducción:

El aprendizaje requiere un estímulo inicial que resulte interesante y nuevo. Precisamente el realizar un análisis del momento actual de la computación, resaltando la evolución de unidades de cómputo en dispositivos con gran capacidad para procesamiento paralelo y conectividad a redes. Así como, buscar la investigación por parte del estudiante de herramientas idóneas para el desarrollo de soluciones y el uso eficaz de los recursos computacionales.

En congruencia con ello, el trabajo final que se propone consiste en la construcción de una solución que implemente una aplicación de programación distribuida y Machine Learning con soporte de Apis, para el problema planteado. El equipo de trabajo utilizará Github como herramienta colaborativa para la gestión del desarrollo de software y Docker compose para el despliegue de la solución.

Objetivo:

El presente documento define el trabajo final y la rúbrica que permite evaluar el logro del curso CC65 Programación Concurrente y Distribuida. El objetivo del trabajo final (TF) es que los estudiantes construyan aplicaciones concurrentes y distribuidas de alto rendimiento de manera eficaz desde el punto de vista de la ingeniería de software.

Logro del curso:

Al finalizar el curso, el estudiante construye aplicaciones concurrentes y distribuidas de alto rendimiento de manera eficaz.

El curso busca desarrollar la competencia general de Razonamiento Cuantitativo en nivel 3 y las competencias específicas Responsabilidad y ética en nivel 2 para Ciencias de computación e Ingeniería de Software.

Instrucciones:

- El trabajo es en equipos de mínimo 2 y máximo 3 estudiantes, se recomienda mantener los grupos formados para el trabajo parcial.
- El trabajo está dividido en 3 entregables.
- Se usará software para detección de plagio.
- El proyecto se aloja en Github y debe seguir Git Flow. Repositorio público.
- El trabajo final será desarrollado íntegramente en GO y contenerizado en Docker.
- Utilizará Docker compose para desplegar la solución, UI, Server API, Nodos.

Alcance del trabajo:

1. Objetivo

El trabajo combina dos grandes habilidades que has aprendido durante el curso: procesar información de forma paralela (varios procesos trabajando al mismo tiempo) y distribuida (varios computadores cooperando), aplicadas a un problema real que beneficia a personas reales.

No se trata solo de escribir código que funcione. Se trata de construir un sistema que sea rápido, robusto y útil, algo de lo que puedas estar orgulloso al mostrarlo fuera del aula.

2. El problema que vas a resolver

Cada día, las ciudades y gobiernos generan enormes cantidades de datos: accidentes de tránsito, lecturas de sensores ambientales, registros de pacientes, consumo de energía eléctrica y muchos más. Estos datos existen, son públicos y gratuitos, pero casi nunca se usan de forma inteligente porque nadie ha construido la herramienta que los analice en tiempo real.

Tu equipo elegirá uno de estos conjuntos de datos y construirá un sistema que:

- Cargue y procese más de un millón de registros de forma concurrente.
- Entrene un modelo de aprendizaje automático de manera distribuida (en varios hilos o procesos).
- Exponga una interfaz de consulta donde cualquier persona pueda hacer preguntas y obtener predicciones.
- Muestre de forma visual el impacto potencial del sistema en la sociedad.

El impacto social no es un requisito vago. Significa que el sistema que construyas debe responder al menos una pregunta que importe a personas fuera de la universidad. Algunos ejemplos concretos:

- ¿En qué horas y zonas hay mayor riesgo de accidente de tránsito para planificar patrullaje preventivo?
- ¿Cuándo se disparará la contaminación del aire para alertar a personas con enfermedades respiratorias?

- ¿En qué hospitales habrá escasez de camas en los próximos días para redistribuir recursos médicos?

Si tu sistema puede responder algo así, tiene impacto social. Esa pregunta debe quedar documentada desde el inicio del proyecto.

3. Visión de Arquitectura

El sistema tiene cuatro partes bien definidas que funcionan como una cadena. Cada parte puede desarrollarse de forma independiente organizado por los miembros del equipo, lo que permite trabajar en paralelo (igual que el código que van a escribir).

Parte A — Cargador de datos concurrente

Es la entrada del sistema. Su trabajo es leer el archivo de datos (que puede tener millones de filas) y procesarlo en paralelo. En lugar de leer línea por línea de forma secuencial, divide el archivo en bloques y asigna cada bloque a un worker diferente.

En Go esto se hace con goroutines y channels. El resultado de esta etapa es un conjunto de datos limpio, validado y listo para entrenar.

Parte B — Motor de entrenamiento distribuido

Es la parte más exigente del proyecto. Toma los datos limpios y entrena un modelo de aprendizaje automático usando múltiples hilos o procesos. El tipo de modelo puede ser simple, como una regresión logística o un árbol de decisión, lo importante no es la complejidad del modelo sino que el entrenamiento ocurra de forma genuinamente paralela.

Aquí se demostrará el conocimiento central del curso: cómo dividir trabajo, cómo sincronizar resultados parciales y cómo garantizar que no haya condiciones de carrera en la actualización de los parámetros del modelo.

Parte C — API de predicciones

Una vez entrenado el modelo, esta parte lo expone como un servicio web. Cualquier persona puede enviar una consulta (por ejemplo, la hora y ubicación de un viaje) y recibir una predicción (por ejemplo, la probabilidad de demora). La API debe responder en menos de 100 milisegundos para ser considerada aceptable.

Parte D — Visualización de impacto

Es la parte más orientada a comunicar los resultados. Puede ser una interfaz SPA, use algún framework por ejemplo: Angular, React con componentes simples o un dashboard con gráficos y métricas. Su función es mostrar, con datos reales, qué problema resuelve el sistema y a cuántas personas podría beneficiar.

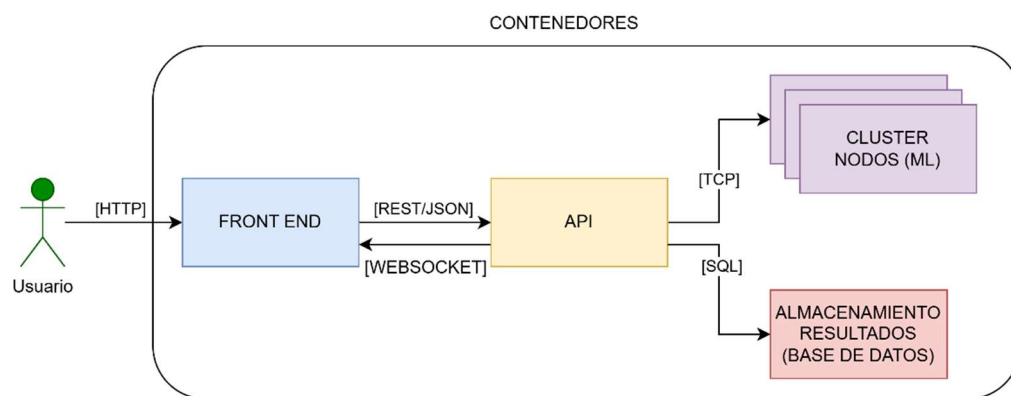


Fig1. Vista de Arquitectura

4. Detalle de la arquitectura

✓ Clúster Nodos (ML):

- Implementa el algoritmo concurrente/distribuido.
- Procesa grandes volúmenes de datos
- Usa goroutines y channels (Go) para dividir y combinar cálculos.
- Comunica resultados al coordinador (API) mediante TCP interno.

✓ API:

- Implementada en Go (recomendado para continuidad con la lógica concurrente).
- Expone endpoints.
- Puede gestionar la autenticación con JWT tokens.
- Hacer de coordinador del cluster, enviando tareas a los nodos de cómputo y unificando resultados.
- Almacenar resultados de recomendaciones en base de datos.
- Servir datos en tiempo real al frontend en formato JSON usando websockets

✓ Base de datos:

- Incluir dos niveles de almacenamiento:
 - Persistente: MongoDB.
 - Cache/colaborativo: Redis: Recomendaciones precalculadas o respuestas parciales.

✓ FrontEnd Web:

- Desarrollar interfaz SPA, use algún framework por ejemplo: Angular, React con componentes simples
- Módulos sugeridos:
 - Inicio de sesión / autenticación.
 - Panel de usuario.
 - Visualización.
 - Panel administrador: métricas del cluster (uso CPU, latencia, número de nodos).

5. Fuentes de datos

Todos los conjuntos de datos listados aquí son de libre uso, están bien documentados y superan el millón de registros. El equipo debe elegir uno y justificar su elección en la propuesta inicial.

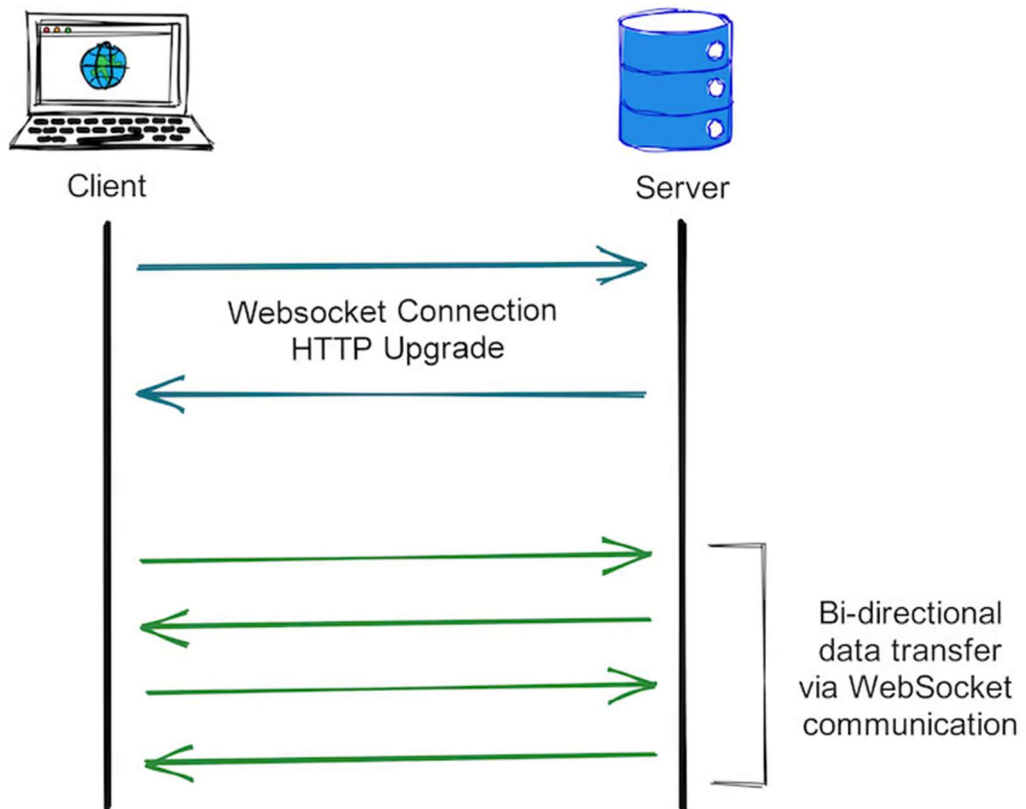
Dataset	Fuente	Registros	Tema social
NYC Taxi Trips	NYC Open Data	>1.5B filas	Movilidad urbana
Calidad del aire (AQI)	OpenAQ / EPA	>500M mediciones	Salud ambiental
Delitos urbanos	data.gov / Kaggle	>4M reportes	Seguridad ciudadana
Registros hospitalarios	UCI ML Repository	>2M registros	Salud pública
Consumo eléctrico	Our World in Data	>1M lecturas	Sostenibilidad

Fuentes adicionales disponibles en: [kaggle.com/datasets](https://www.kaggle.com/datasets), data.gov, opendata.cityofnewyork.us, archive.ics.uci.edu. Todos tienen licencia de uso libre para proyectos académicos.

6. Anexo

La especificación del protocolo WebSocket define dos esquemas URI:

- WebSocket (ws): used for non-encrypted connections
- WebSocket Secure (wss): used for encrypted connections



Evaluación del Trabajo Final

Instrucciones de Entrega:

- Código fuente de la solución eliminando cualquier defecto.
- Incluir las imágenes y demás recursos utilizados para la elaboración de la solución.
- Video de presentación de la solución, puntos importantes de su construcción y su funcionamiento (end to end), realizado en un máximo de 6 minutos donde participen todos los integrantes y demuestren el conocimiento del tema.
- El trabajo será entregado por cada integrante del grupo y mediante el aula virtual.
- El plazo es impostergable y por ningún motivo y/o circunstancia se recibirá trabajos fuera de esa fecha y hora, ni por otro medio diferente al aula virtual.

Plazos de Entrega:

- Entregable 1(PC3) – Fecha de entrega semana 11 (12/06/2026 23:30 Hrs.)
 - Elaborar informe de los puntos:
 - Presentación del caso a resolver (Problema y motivación)
 - Limpieza y Análisis de datos
 - Diseño del modelo ML
 - Paralelización del cálculo
 - Evidencias de la implementación
 - Reporte de participación
- Entregable 2(PC4) – Fecha de entrega semana 13 (26/06/2026 23:30 Hrs.)
 - Elaborar informe de los puntos:
 - Distribución (Cluster de nodos ML)
 - Desarrollo de API
 - Implementación de Bases de datos
 - Evidencias de la implementación y pruebas de funcionamiento
 - Documentación (el informe debe contener el entregable 3 con las correcciones)
 - Reporte de participación
- Entregable 3(TB2) – Fecha de entrega semana 15 (05/07/2026 23:30 Hrs.)
 - Elaborar informe de los puntos:
 - Desarrollo FrontEnd Web
 - Evaluación experimental
 - Documentación y Presentación Final (el informe final contiene los entregables 3 y 4 con las correcciones)
 - Reporte de participación

Detalle de Entrega:

1. El archivo ZIP o RAR a presentar tendrá por nombre:
1ACC0065_YYY_UXXXXXXXXX, en donde los caracteres YYY reemplaza el código de entregable (PC3, PC4, TB2), las X se reemplazarán por el código de alumno.
2. El informe del trabajo debe tener el nombre con el siguiente formato:
1ACC0065_YYY_202610_Informe_UXXXXXXXXX
3. Solo se calificarán los trabajos entregados mediante el Aula virtual.

4. Estamos seguros de que cada uno realizará su trabajo, sin embargo, para evitar cualquier perspicacia, [le recomendamos leer el reglamento de disciplina del alumno, en el cual se indican las faltas y las sanciones que se indican en el caso de haber copia de trabajos.](#)

El informe Final debe incluir los siguientes puntos:

- a) Carátula
- b) Resumen
- c) Índice
- d) Descripción del problema y motivación
- e) Objetivos
- f) Desarrollo (Evidencias de la implementación de cada etapa y arquitectura)
- g) Conclusiones
- h) Recomendaciones
- i) Bibliografía (utilizar APA7)
- j) Anexos (Link de repositorio GitHub, link de video, documentos, informes, otros)

Links de ayuda

Proyectos en GitHub:

- https://youtu.be/Vjf_s7TGmqY

Rúbrica

	Sobresaliente	En Proceso	Deficiente
Planteamiento	3 puntos Definición del problema, dataset elegido. Usa diagrama de arquitectura solución detallada para complementar la explicación de su planteamiento solución. Programación concurrente y distribuida. Tiene en cuenta las restricciones del trabajo. Cumple con la exposición de su trabajo demostrando	2 puntos Define procesos e Información incompleta. Expone y no demuestra conocimiento total del tema.	0 puntos No elaborado

	conocimiento total del tema.		
Implementación	8 puntos Carga, limpieza y procesamiento concurrente de >1M de registros. Entrenamiento paralelo del modelo ML usando goroutines. Funciona correctamente, utiliza docker, puertos, para procesar los datos generados por la solución concurrentemente, usa canales y muestra resultados siguiendo las consideraciones de la arquitectura solución. Usa Apis para exponer métodos que consuma el front end y recopile los resultados del cluster. Servicio REST con predicciones en tiempo real y métricas de rendimiento. Tiene en cuenta las restricciones del trabajo. Cumple con la exposición de su trabajo demostrando conocimiento total del tema.	5 puntos Elabora el código y ejecuta concurrentemente, usa puertos, canales, muestra resultados, implementando las consideraciones del trabajo de forma parcial. Expone y demuestra conocimiento parcial del tema.	0 puntos No elaborado
Interfaz	5 puntos La interfaz en modo gráfico usando un framework UI, muestra un menú de opciones, configura el aplicativo con parámetros ingresados desde la UI, muestra los	3 puntos Muestra el funcionamiento de la aplicación y resultados entendible en página web parcialmente. Expone y demuestra conocimiento parcial del tema.	0 puntos No elaborado

	resultados en tiempo real, demuestra una adecuada experiencia de usuario y organizada en página SPA que muestre resultados e impacto social. Tiene en cuenta las restricciones del trabajo. Cumple con la exposición de su trabajo demostrando conocimiento total del tema.		
Informe	2 puntos Hace buen uso del medio escrito, cumple con la estructura del informe, guarda coherencia y presenta resultados, conclusiones entendibles y orientadas a resultados. Anexa el historial del versionamiento del código fuente, evidencia el git Flow. Cumple con el reporte de participación. Cumple con la exposición de su trabajo demostrando conocimiento total del tema.	1 puntos Sigue una estructura clara y contiene los elementos mínimos necesarios solicitados. Usa herramienta de versionado de código. Informe incompleto. Expone y demuestra conocimiento parcial del tema.	0 puntos No elaborado, No demuestra conocimiento del tema.
Video	2 puntos Elabora un video de 6 minutos como máximo donde el grupo presenta la solución, puntos importantes de su construcción y su funcionalidad (end to end), demostrando dominio del tema.	1 puntos Cubre los puntos solicitados de forma parcial.	0 puntos No elaborar el video afecta la calificación en los demás ítems de la rúbrica.

Santiago de Surco, mayo de 2026